

CLBS: Central Load Balancing Scheduler pour Proxmox VE

Auteur : [Votre nom]

Date : 2025

Version : 1.0

Table des matières

1. [Introduction et contexte](#)
2. [Architecture générale](#)
3. [Acquisition des données : deux sources complémentaires](#)
4. [L'algorithme CSLB : les cinq phases](#)
5. [Mécanismes de sécurité](#)
6. [Persistance des données : SQLite](#)
7. [Exposition des métriques : endpoint Prometheus](#)
8. [Journalisation \(Logs\)](#)
9. [Configuration et déploiement](#)
10. [Mise en place de l'infrastructure : InfluxDB et clé API Proxmox](#)
11. [Bibliographie](#)

1. Introduction et contexte

Problème posé

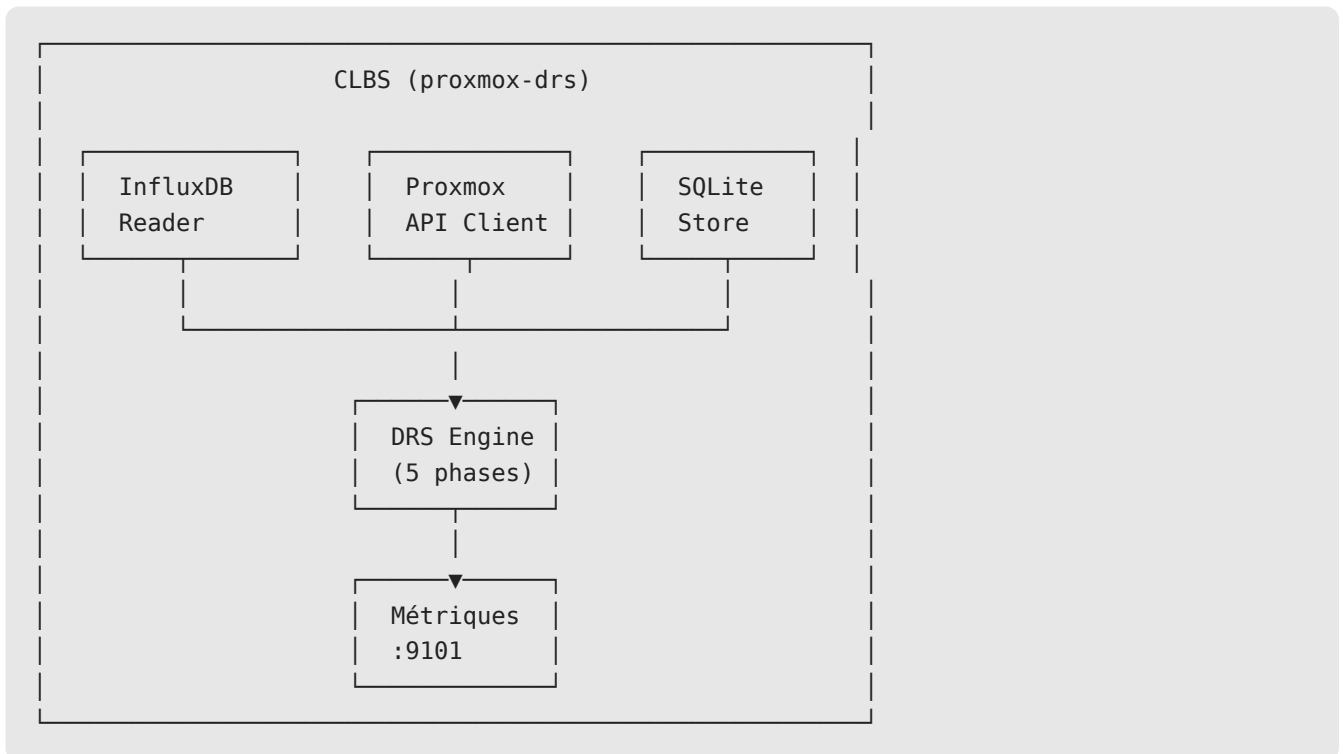
Dans un cluster Proxmox VE multi-nœuds, la charge des machines virtuelles (VMs) et des conteneurs (CTs) ne se répartit pas naturellement de façon uniforme. Certains nœuds se retrouvent saturés en CPU pendant que d'autres tournent presque à vide. Sans intervention, cette situation dégrade les performances des VMs hébergées sur les nœuds les plus chargés et gaspille les ressources des nœuds légers.

Solution : CLBS

CLBS (Central Load Balancing Scheduler) est un service interne développé en Go qui implémente un moteur **DRS** (Dynamic Resource Scheduling) inspiré du système éponyme de VMware vSphere. Il évalue en continu la charge de chaque nœud du cluster, calcule les décisions de migration selon l'algorithme **CSLB** (Central Scheduler Load Balancing) décrit par Chandak et al. (2012), et déclenche des migrations à chaud via l'API Proxmox pour maintenir l'équilibre du cluster.

Le service est un binaire Go unique, sans dépendance à des frameworks externes lourds. Il s'exécute en tant qu'unité systemd sur une machine dédiée (`192.168.10.104`) et tourne en boucle permanente, un cycle toutes les 60 secondes.

2. Architecture générale



Le moteur DRS s'appuie sur trois composants :

- **InfluxReader** : lit les métriques CPU des VMs et CTs depuis InfluxDB (Telegraf agent).
- **ProxmoxAPI** : interroge le cluster pour obtenir l'état des nœuds et des guests, et déclenche les migrations.
- **Store** : persiste l'historique des migrations dans une base SQLite locale afin de survivre aux redémarrages du service.

3. Acquisition des données : deux sources complémentaires

Pourquoi deux sources ?

L'API Proxmox fournit des métriques agrégées au niveau du nœud (CPU global, RAM globale), mais elle **ne fournit pas** de métriques CPU fiables par VM ou par CT en temps réel de façon facilement interrogeable. Pour sélectionner quelle VM migrer, CLBS a besoin de connaître la consommation CPU individuelle de chaque guest. C'est pourquoi **Telegraf** est déployé sur chaque nœud Proxmox pour collecter ces métriques et les stocker dans **InfluxDB**, ce qui permet de les interroger par plage de temps avec le langage Flux.

3.1 L'API Proxmox

CLBS interroge l'API REST Proxmox via HTTPS avec vérification TLS désactivée (certificat auto-signé). L'authentification se fait par **API Token** au format `PVEAPIToken=user@realm!tokenname=<secret>`.

```
func newProxmoxAPI() *ProxmoxAPI {
    transport := &http.Transport{
        TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
    }
    return &ProxmoxAPI{
        base: strings.TrimRight(pveHost, "/"),
    }
}
```

```

    client: &http.Client{Transport: transport, Timeout: 30 * time.Second},
    token:  fmt.Sprintf("PVEAPIToken=%s=%s", pveTokenID, pveTokenSec),
  }
}

```

Toutes les requêtes HTTP passent par un wrapper générique qui extrait le champ `data` de l'enveloppe JSON que Proxmox retourne systématiquement :

```

func (p *ProxmoxAPI) get(path string) (json.RawMessage, error) {
    req, _ := http.NewRequest("GET", p.base+"/api2/json"+path, nil)
    req.Header.Set("Authorization", p.token)
    // ...
    var envelope struct {
        Data json.RawMessage `json:"data"`
    }
    json.Unmarshal(body, &envelope)
    return envelope.Data, nil
}

```

Les endpoints Proxmox utilisés par CLBS sont :

Endpoint	Méthode	Usage
<code>/api2/json/nodes</code>	GET	Liste et état de tous les nœuds du cluster
<code>/api2/json/nodes/{node}/qemu</code>	GET	Liste des VMs (QEMU) sur un nœud
<code>/api2/json/nodes/{node}/lxc</code>	GET	Liste des conteneurs LXC sur un nœud
<code>/api2/json/nodes/{node}/qemu/{vmid}/status/current</code>	GET	Mémoire maximale allouée à une VM
<code>/api2/json/nodes/{node}/lxc/{vmid}/status/current</code>	GET	Mémoire maximale allouée à un CT
<code>/api2/json/nodes/{node}/qemu/{vmid}/migrate</code>	POST	Déclenche la migration à chaud d'une VM
<code>/api2/json/nodes/{node}/lxc/{vmid}/migrate</code>	POST	Déclenche la migration à chaud d'un CT

Données récupérées depuis l'API Proxmox :

- CPU global du nœud (ratio 0.0 à 1.0, converti en pourcentage)
- RAM utilisée et RAM totale du nœud (en octets)
- Liste des VMs et CTs avec leur statut (`running`, `stopped`, etc.)
- Mémoire maximale allouée à un guest spécifique (pour le test de capacité du nœud destination)

Ce que l'API Proxmox ne fournit pas facilement : la consommation CPU individuelle par VM sur une fenêtre de temps glissante et cohérente. C'est la lacune que InfluxDB comble.

3.2 InfluxDB via Telegraf

Telegraf est déployé sur chaque nœud Proxmox avec le plugin `proxmox` activé. Il collecte les métriques par guest et les pousse dans un bucket InfluxDB. CLBS interroge ensuite ce bucket en Flux pour obtenir la

moyenne CPU de chaque VM sur les **15 dernières minutes**, ce qui lisse les pics courts et donne une image représentative de la charge réelle.

Métriques collectées par Telegraf depuis Proxmox :

- `_measurement = "system"`, `_field = "cpu"` : utilisation CPU du guest (ratio 0.0 à 1.0)
- Tags pertinents : `host` (nom du guest), `nodename` (nœud qui héberge le guest), `object` (`"qemu"` ou `"lxc"`)

La requête Flux pour les VMs QEMU sur un nœud donné :

```
from(bucket: "proxmox")
  |> range(start: -15m)
  |> filter(fn: (r) => r._measurement == "system")
  |> filter(fn: (r) => r._field == "cpu")
  |> filter(fn: (r) => r["object"] == "qemu")
  |> filter(fn: (r) => r["_value"] > 0)
  |> filter(fn: (r) => r["nodename"] == "pve-node1")
  |> group(columns: ["host"])
  |> mean()
```

La valeur retournée est un ratio entre 0 et 1, multiplié par 100 pour obtenir un pourcentage :

```
func (ir *InfluxReader) GetVMCPU(nodeName string) map[string]float64 {
    // ... requête Flux ...
    for host, val := range raw {
        result[host] = math.Round(val * 100 * 100) / 100
    }
    return result
}
```

Une requête identique est faite pour les conteneurs LXC (`object == "lxc"`).

4. L'algorithme CSLB : les cinq phases

Le moteur DRS exécute un cycle complet toutes les **60 secondes**. Chaque cycle suit strictement les cinq phases de l'algorithme CSLB.

Vue d'ensemble du cycle

```
[Démarrage du cycle]
  |
  ▼
[Garde anti-tempête] — tempête active → STOP (pause 30 min)
  |
  ▼ pas de tempête
[Phase 1 : Évaluation de la charge]
  - Récupère CPU/RAM de chaque nœud via API Proxmox
  - Calcule le seuil (threshold) = moyenne CPU de tous les nœuds
  - Classe chaque nœud : light / moderate / heavy
  |
  ▼
[Phase 2 : Contrôle de rentabilité]
```

```

- Existe-t-il au moins un nœud "heavy" ET un nœud "light" ?
- Non ► STOP (cluster équilibré)
  |
  ▼ Oui
[Contrôle cooldown global]
- Une migration a eu lieu il y a moins de 300s ? ► STOP
  |
  ▼
Pour chaque nœud "heavy" :
  |
  ▼
[Phase 3 : Work Transfer Vector (WTV)]
- WTV = CPU_heavy - threshold
  |
  ▼
[Phase 4 : Sélection du guest + destination]
- Interroge InfluxDB pour le CPU individuel de chaque guest
- Score = |CPU_guest - WTV| (les VMs QEMU ont un bonus de -200)
- Sélectionne le guest avec le score le plus bas
- Trouve le nœud "light" qui peut absorber la charge
  |
  ▼
[Phase 5 : Migration]
- Appel POST sur l'API Proxmox (migration à chaud, online=1)
- Enregistrement dans SQLite et métriques
- Une seule migration par cycle, puis break
  |
  ▼
[Fin du cycle - attente 60s]

```

Phase 1 : Évaluation de la charge (Load Evaluation)

CLBS récupère l'état de tous les nœuds du cluster via l'API Proxmox et calcule les bandes de charge selon la formule du papier CSLB :

```

threshold = moyenne(CPU_i pour tous les nœuds)
mean      = (CPU_min + CPU_max) / 2
diff      = |threshold - mean|
half_width = max(diff, 10.0) ← minimum de 10 points de pourcentage
lower     = threshold - half_width
upper     = threshold + half_width

```

Classification :

- CPU > upper → **heavy** (surchargé)
- CPU < lower → **light** (sous-chargé)
- sinon → **moderate** (équilibré)

```

func (bc *BandCalculator) Compute(nodes []*NodeLoad) (threshold, lower, upper float64) {
    cpuMin, cpuMax, cpuSum := nodes[0].CPUPct, nodes[0].CPUPct, 0.0
    for _, n := range nodes {
        cpuSum += n.CPUPct
        if n.CPUPct < cpuMin { cpuMin = n.CPUPct }
        if n.CPUPct > cpuMax { cpuMax = n.CPUPct }
    }
    threshold = cpuSum / float64(len(nodes))

```

```

meanVal    := (cpuMin + cpuMax) / 2.0
diff       := math.Abs(threshold - meanVal)
halfWidth  := math.Max(diff, minBandWidth) // minBandWidth = 10.0

lower = threshold - halfWidth
upper = threshold + halfWidth
// ...
}

```

Phase 2 : Contrôle de rentabilité (Profitability Check)

La migration n'a de sens que s'il existe simultanément un nœud `heavy` (à décharger) et un nœud `light` (pour accueillir le guest). Si le cluster est déjà équilibré (tous les nœuds en `moderate`), le cycle s'arrête immédiatement sans action.

```

func (e *DRSEngine) isProfitable(nodes []*NodeLoad) bool {
    var hasHeavy, hasLight bool
    for _, n := range nodes {
        if n.Band == "heavy" { hasHeavy = true }
        if n.Band == "light" { hasLight = true }
    }
    return hasHeavy && hasLight
}

```

Phase 3 : Work Transfer Vector (WTV)

Le WTV représente la quantité de CPU à "déplacer" hors d'un nœud `heavy` pour le ramener au seuil :

```
WTV = CPU_heavy - threshold
```

```

func (e *DRSEngine) workTransferVector(heavy *NodeLoad, threshold float64) float64 {
    return math.Round((heavy.CPUPct - threshold) * 100) / 100
}

```

Ce vecteur sert de cible : on cherche le guest dont la consommation CPU est la plus proche de cette valeur, ce qui maximise l'efficacité de la migration en un seul mouvement.

Phase 4 : Sélection du guest et de la destination

CLBS interroge InfluxDB pour obtenir le CPU moyen des 15 dernières minutes de chaque guest sur le nœud `heavy`. Ensuite, pour chaque guest candidat, un score est calculé :

```

score = |CPU_guest - WTV|           (pour les VMs QEMU)
score = |CPU_guest - WTV| + 200     (pour les CTs LXC, pénalité de priorité)

```

Les VMs QEMU sont préférées aux CTs LXC car leur migration à chaud est plus robuste. Le guest avec le **score le plus bas** est sélectionné en premier.

```

for _, v := range vms {
    cpu := vmCPU[v.Name]
    diff := math.Abs(cpu - wtv)
    candidates = append(candidates, scored{

```

```

        score: diff,          // pas de pénalité pour QEMU
        diff: diff,
        guest: v,
        kind: "qemu",
        cpu: cpu,
    })
}
for _, c := range cts {
    cpu := ctCPU[c.Name]
    diff := math.Abs(cpu - wtv)
    candidates = append(candidates, scored{
        score: diff + vmPriorityBonus, // +200 pour LXC
        diff: diff,
        guest: c,
        kind: "lxc",
        cpu: cpu,
    })
}
}

```

Sélection du nœud destination :

Une fois le guest sélectionné, CLBS cherche le meilleur nœud `light` selon un score composite :

```
score_destination = 0.7 × CPU_dest + 0.3 × RAM_dest
```

Le nœud avec le **score composite le plus bas** est préféré. Avant de valider le choix, une vérification de capacité est effectuée : la charge projetée après migration (CPU actuel du dest + CPU du guest, RAM actuelle + RAM allouée au guest) ne doit pas dépasser les seuils définis.

```

func (e *DRSEngine) destinationCanHold(dest *NodeLoad, vm *VMInfo, upper float64) bool {
    projCPU := dest.CPUPct + vm.CPUPct
    vmRAMPct := (vm.RAMMaxMB / dest.RAMTotalMB) * 100.0
    projRAM := dest.RAMPct + vmRAMPct

    if projCPU > upper { return false } // dépasserait la bande moderate
    if projRAM >= ramHigh { return false } // dépasserait le cap RAM (85%)
    return true
}

```

Phase 5 : Migration à chaud

La migration est déclenchée via l'API Proxmox. Elle se fait **en ligne** (`online: 1`), ce qui signifie que le guest continue de fonctionner pendant le transfert. Pour les VMs QEMU, les disques locaux ne sont pas copiés (`with-local-disks: 0`) car le stockage partagé est supposé accessible depuis tous les nœuds.

```

func (p *ProxmoxAPI) MigrateVM(vmid int, src, dst string) (json.RawMessage, error) {
    return p.post(
        fmt.Sprintf("/nodes/%s/qemu/%d/migrate", src, vmid),
        map[string]any{
            "target": dst,
            "online": 1,
            "with-local-disks": 0,
        },
    )
}

```

```
func (p *ProxmoxAPI) MigrateCT(vmid int, src, dst string) (json.RawMessage, error) {
    return p.post(
        fmt.Sprintf("/nodes/%s/lxc/%d/migrate", src, vmid),
        map[string]any{
            "target": dst,
            "online": 1,
        },
    )
}
```

Après une migration réussie, l'événement est persisté dans SQLite et enregistré dans les métriques en mémoire. **Une seule migration est effectuée par cycle**, puis le cycle se termine et le cooldown de 300 secondes s'applique.

5. Mécanismes de sécurité

L'algorithme CSLB seul pourrait provoquer des migrations en cascade ou des oscillations (une VM qui va et vient entre deux nœuds). CLBS implémente plusieurs gardes-fous :

5.1 Cooldown global post-migration

Après toute migration, aucune nouvelle migration n'est autorisée pendant **300 secondes** (5 minutes). Cela laisse au cluster le temps de se stabiliser avant la prochaine évaluation.

```
const migrationCooldown = 300 * time.Second

func (e *DRSEngine) cooldownOK() bool {
    return time.Since(e.lastMigration) >= migrationCooldown
}
```

5.2 Cooldown par VM (anti-oscillation)

Si la même VM est migrée **2 fois ou plus en 11 minutes** (fenêtre de 660 secondes), elle entre en cooldown individuel de **30 minutes**. Cela empêche une VM dont la charge fluctue rapidement d'être migrée en boucle.

```
const vmDoubleMigrationWindow = 660.0 // secondes
const vmDoubleMigrationCooldown = 1800.0 // secondes (30 minutes)

func (e *DRSEngine) vmCooldownOK(vmid int, name string) bool {
    // compte les migrations récentes de cette VM dans la fenêtre
    if len(recent) >= 2 {
        // applique le cooldown de 30 minutes
        return false
    }
    return true
}
```

5.3 Détection de tempête de migrations

Si **6 migrations ou plus** se produisent dans une fenêtre de **31 minutes** (1860 secondes), le moteur DRS se suspend automatiquement pendant **30 minutes**. Ce seuil signale une situation anormale qui nécessite une revue manuelle du cluster.

```
const stormWindow      = 1860.0 // ~31 minutes
const stormThreshold    = 6
const stormPause       = 1800.0 // 30 minutes

func (e *DRSEngine) checkMigrationStorm() bool {
    if len(e.clusterMigrations) >= stormThreshold {
        e.stormPauseUntil = now.Add(30 * time.Minute)
        logger.Critical("MIGRATION STORM DETECTED - pause DRS 30min")
        return true
    }
    return false
}
```

5.4 Cap RAM dur

Un nœud destination dont la RAM dépasse **85%** est systématiquement rejeté, indépendamment de son score CPU. Cela évite de migrer vers un nœud qui serait sur le point de faire du swap.

```
const ramHigh = 85.0 // %
```

5.5 Largeur de bande minimale

Les bandes sont forcément larges d'au moins **10 points de pourcentage CPU** (5 de chaque côté du seuil). Si le cluster est peu chargé et que les nœuds sont tous proches de la moyenne, les bandes seraient trop étroites et provoqueraient des migrations pour des différences négligeables. Ce garde-fou évite les migrations inutiles.

```
const minBandWidth = 10.0 // %
```

5.6 Exclusions statiques

Certaines VMs et CTs critiques sont explicitement exclues de toute migration, définies dans le code source :

```
var (
    excludedVMs      = map[string]bool{"monitoring": true, "OPNsense": true, "gitlab": true,
    "clbs-services": true}
    excludedVMIDs     = map[int]bool{100: true, 102: true, 103: true, 104: true}
    excludedCTs       = map[string]bool{"monitoring": true}
    excludedCTIDs     = map[int]bool{102: true}
)
```

De plus, la VM tout juste migrée lors du cycle précédent est protégée pour le cycle suivant (`lastMigratedVMID`), évitant qu'elle soit immédiatement remigrée.

Tableau récapitulatif

Mécanisme	Valeur	Description
Cooldown global	300 s	Pause après toute migration

Mécanisme	Valeur	Description
Cooldown par VM	fenêtre 660 s / pause 1800 s	2 migrations en 11 min bloque la VM 30 min
Détection de tempête	6 migrations / 1860 s	Pause DRS 30 min, revue manuelle requise
Cap RAM dur	85%	Nœud dest rejeté si RAM > 85%
Largeur de bande minimale	10%	Pas d'action si l'écart est trop faible
Exclusions statiques	Listes dans <code>main.go</code>	VMs et CTs jamais migrés

6. Persistance des données : SQLite

CLBS maintient une base SQLite à `/opt/proxmox-drs/clbs.db` pour survivre aux redémarrages du service. Sans persistance, un redémarrage remettrait à zéro tous les cooldowns et permettrait des migrations abusives juste après un restart.

Schéma

```
CREATE TABLE IF NOT EXISTS migrations (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  at          INTEGER NOT NULL,
  migrated_at TEXT NOT NULL DEFAULT '',
  vmid        INTEGER NOT NULL,
  vm_name     TEXT NOT NULL,
  kind        TEXT NOT NULL,
  src         TEXT NOT NULL,
  dst         TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_migrations_at ON migrations(at);
```

Comportement au démarrage

Au lancement, CLBS charge les migrations des **72 dernières heures** depuis SQLite et reconstruit en mémoire :

- l'historique des migrations par VM (pour les cooldowns individuels)
- la liste des migrations dans la fenêtre de tempête (pour le storm guard)
- le timestamp et le VMID de la dernière migration (pour le cooldown global)

```
if events, err := store.LoadRecent(); err == nil {
  metrics.LoadMigrations(events)
  for _, ev := range events {
    e.migrationHistory[ev.VMID] = append(e.migrationHistory[ev.VMID], ev.At)
    e.clusterMigrations = append(e.clusterMigrations, ev.At)
  }
}
```

Les enregistrements de plus de 72 heures sont purgés automatiquement à chaque insertion.

7. Exposition des métriques : endpoint Prometheus

CLBS expose un endpoint HTTP compatible Prometheus sur le port **9101** :

```
GET http://<clbs-host>:9101/metrics    # format texte Prometheus
GET http://<clbs-host>:9101/healthz    # retourne "ok"
```

Les métriques sont écrites par le moteur DRS à chaque cycle et lues par le handler HTTP à chaque scrape, avec une protection par `sync.RWMutex`.

Métriques disponibles

Métrique	Type	Description
<code>clbs_cycles_total</code>	counter	Total de cycles DRS exécutés
<code>clbs_cycle_errors_total</code>	counter	Cycles ayant échoué ou paniqué
<code>clbs_cycle_duration_seconds</code>	gauge	Durée du dernier cycle
<code>clbs_last_cycle_timestamp</code>	gauge	Timestamp Unix du dernier cycle
<code>clbs_migrations_total</code>	counter	Total de migrations déclenchées
<code>clbs_last_migration_timestamp</code>	gauge	Timestamp Unix de la dernière migration
<code>clbs_migrations_in_storm_window</code>	gauge	Migrations dans la fenêtre anti-tempête
<code>clbs_band_threshold_percent</code>	gauge	Seuil CPU calculé (moyenne de tous les nœuds)
<code>clbs_band_lower_percent</code>	gauge	Borne inférieure de la bande modérée
<code>clbs_band_upper_percent</code>	gauge	Borne supérieure de la bande modérée
<code>clbs_ram_hard_cap_percent</code>	gauge	Cap RAM statique (85%)
<code>clbs_node_cpu_percent{node}</code>	gauge	CPU en % par nœud Proxmox
<code>clbs_node_ram_percent{node}</code>	gauge	RAM en % par nœud Proxmox
<code>clbs_node_band{node}</code>	gauge	Classification : 0=light, 1=moderate, 2=heavy
<code>clbs_storm_pause_active</code>	gauge	1 si la pause tempête est active
<code>clbs_storm_pause_remaining_seconds</code>	gauge	Secondes restantes dans la pause tempête
<code>clbs_global_cooldown_active</code>	gauge	1 si le cooldown global est actif
<code>clbs_global_cooldown_remaining_seconds</code>	gauge	Secondes restantes dans le cooldown global
<code>clbs_migration_info{vmid,vm,kind,src,dst,migrated_at}</code>	gauge	Migrations récentes (72h), valeur = 1

Ces métriques peuvent être scrapées par Prometheus et visualisées dans Grafana pour créer un tableau de bord de supervision du cluster.

8. Journalisation (Logs)

CLBS écrit tous ses logs sur `stderr`, capturé par journald sous systemd. Les logs sont colorisés quand le processus est attaché à un terminal (couleurs ANSI).

Niveaux de log

Niveau	Couleur	Usage
DEBUG	Cyan	Détails de débogage fins
INFO	Vert	Progression normale du cycle, décisions prises
WARNING	Jaune	Situations non bloquantes (nœud à passer, cooldown, etc.)
ERROR	Rouge	Erreurs récupérables (échec d'une requête API, migration échouée)
CRITICAL	Rouge gras	Situations critiques : tempête de migrations, variables manquantes au démarrage

Format des logs

```
2025-01-15 14:32:01 [INFO] DRS cycle starting
2025-01-15 14:32:01 [INFO] Band calc: threshold=42.3% moderate=[32.3%, 52.3%]
2025-01-15 14:32:01 [INFO]   pve-node1: CPU=68.5% RAM=71.2% band=heavy guests=8
2025-01-15 14:32:01 [INFO]   pve-node2: CPU=12.1% RAM=45.6% band=light guests=3
2025-01-15 14:32:01 [INFO]   pve-node3: CPU=38.9% RAM=52.0% band=moderate guests=5
2025-01-15 14:32:01 [WARNING] Heavy node: pve-node1 CPU=68.5% RAM=71.2%
2025-01-15 14:32:01 [INFO] Work Transfer Vector for pve-node1: 26.2%
2025-01-15 14:32:01 [INFO] Evaluating 5 VMs + 2 CTs on pve-node1 against WTV=26.2%
2025-01-15 14:32:01 [INFO]   [VM] webapp-prod (vmid=201): cpu=24.1% diff_to_wtv=2.1%
score=2.1
2025-01-15 14:32:01 [INFO]   [VM] db-replica (vmid=205): cpu=31.0% diff_to_wtv=4.8%
score=4.8
2025-01-15 14:32:01 [INFO] Selected [VM] webapp-prod (201) cpu=24.1% score=2.1
2025-01-15 14:32:01 [INFO] Destination: pve-node2 score=0.22 cpu=12.1% ram=45.6%
2025-01-15 14:32:01 [INFO] Plan: move [QEMU] webapp-prod (201) from pve-node1 to pve-
node2
2025-01-15 14:32:02 [INFO] Migration task submitted: "UPID:pve-node1:..."
```

Consulter les logs en production

```
# Suivre les logs en temps réel
journalctl -u proxmox-drs -f

# Afficher les 200 dernières lignes
journalctl -u proxmox-drs -n 200

# Filtrer uniquement les erreurs et critiques
journalctl -u proxmox-drs -p err..crit
```

9. Configuration et déploiement

Variables d'environnement

CLBS charge sa configuration depuis un fichier `.env` dans le répertoire de travail (ou directement depuis les variables d'environnement) :

```
# Proxmox VE
PVE_HOST=https://192.168.10.1:8006
PVE_TOKEN_ID=root@pam!drs
PVE_TOKEN_SEC=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx

# InfluxDB
INFLUX_URL=http://192.168.10.x:8086
INFLUX_TOKEN=<token influxdb>
INFLUX_ORG=<organisation>
INFLUX_BUCKET=proxmox
```

Toutes les variables sont obligatoires. Le service refuse de démarrer si l'une d'elles est absente, avec un message `CRITICAL` explicite.

Unité systemd

```
[Unit]
Description=Proxmox DRS - CSLB Load Balancer
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
WorkingDirectory=/opt/proxmox-drs
EnvironmentFile=/opt/proxmox-drs/.env
ExecStart=/opt/proxmox-drs/proxmox-drs
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Build et déploiement

```
# Compiler le binaire
make build

# Déployer manuellement
sudo systemctl stop proxmox-drs
sudo cp proxmox-drs /opt/proxmox-drs/proxmox-drs
sudo cp .env /opt/proxmox-drs/.env
sudo systemctl start proxmox-drs

# Vérifier le statut
```

```
systemctl status proxmox-drs
journalctl -u proxmox-drs -f
```

Un script `update_clbs.sh` automatise ces étapes :

```
#!/bin/bash
make build
systemctl stop proxmox-drs
cp proxmox-drs /opt/proxmox-drs/proxmox-drs
cp .env /opt/proxmox-drs/.env
systemctl start proxmox-drs
```

10. Mise en place de l'infrastructure : InfluxDB et clé API Proxmox

10.1 Création du bucket InfluxDB et configuration de Telegraf

Étape 1 : Créer un bucket dans InfluxDB

Depuis l'interface web InfluxDB (`http://<influx-host>:8086`) :

1. Aller dans **Data > Buckets > Create Bucket**
2. Nommer le bucket (ex. `proxmox`)
3. Définir la rétention (ex. 30 jours)
4. Cliquer sur **Create**

Étape 2 : Générer un token InfluxDB

1. Aller dans **Data > API Tokens > Generate API Token**
2. Choisir **Custom API Token** ou **All Access Token** selon le niveau souhaité
3. Copier le token généré dans la variable `INFLUX_TOKEN` du fichier `.env`

Étape 3 : Installer et configurer Telegraf sur chaque nœud Proxmox

```
# Sur chaque nœud Proxmox (Debian/Ubuntu)
apt-get install telegraf
```

Fichier de configuration Telegraf (`/etc/telegraf/telegraf.conf`) pour le plugin Proxmox :

```
[agent]
interval = "30s"
hostname = "pve-node1" # adapter par nœud

[[outputs.influxdb_v2]]
urls = ["http://<influx-host>:8086"]
token = "<INFLUX_TOKEN>"
organization = "<INFLUX_ORG>"
bucket = "proxmox"

[[inputs.proxmox]]
base_url = "https://localhost:8006/api2/json"
api_token = "telegraf@pve!telegraf=<token>"
```

```
insecure_skip_verify = true
node_name = "pve-node1" # adapter par nœud
```

```
systemctl enable telegraf
systemctl start telegraf
```

Pour vérifier que les données arrivent dans InfluxDB, exécuter la requête Flux suivante dans l'interface web InfluxDB :

```
from(bucket: "proxmox")
  |> range(start: -5m)
  |> filter(fn: (r) => r._measurement == "system")
  |> filter(fn: (r) => r._field == "cpu")
  |> limit(n: 10)
```

10.2 Création de la clé API Proxmox pour CLBS

CLBS a besoin d'un API Token Proxmox avec des droits suffisants pour lister les nœuds, les VMs, les CTs, et déclencher des migrations.

Étape 1 : Créer un utilisateur dédié (recommandé)

Dans l'interface web Proxmox, aller dans **Datacenter** > **Permissions** > **Users** > **Add** :

- User name : `drs`
- Realm : `pve` (authentification PVE interne)

Étape 2 : Créer l'API Token

Dans **Datacenter** > **Permissions** > **API Tokens** > **Add** :

- User : `drs@pve`
- Token ID : `clbs`
- Décocher "Privilege Separation" pour que le token hérite des droits de l'utilisateur

Cela génère un token de la forme : `drs@pve!clbs=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`

Étape 3 : Attribuer les permissions

Dans **Datacenter** > **Permissions** > **Add** > **User Permission** :

- Path : `/` (toute la ressource, ou restreindre à `/nodes` selon la politique de sécurité)
- User : `drs@pve`
- Role : `PVEAdmin` (ou un rôle personnalisé avec au minimum `VM.Migrate`, `VM.Monitor`, `Sys.Audit`)

Étape 4 : Renseigner dans `.env`

```
PVE_HOST=https://<ip-proxmox>:8006
PVE_TOKEN_ID=drs@pve!clbs
PVE_TOKEN_SEC=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

Vérification : tester l'accès API manuellement depuis la machine CLBS :

```
curl -sk -H "Authorization: PVEAPIToken=drs@pve!clbs=<secret>" \
https://<pve-host>:8006/api2/json/nodes | python3 -m json.tool
```

11. Bibliographie

- Chandak, A., Jaju, K., Kanfade, A., Lohiya, P., et Joshi, A. (2012, mai). « Dynamic load balancing of virtual machines using QEMU-KVM ». *International Journal of Computer Applications*, 46(6), 10-14.
<https://research.ijcaonline.org/volume46/number6/pxc3879263.pdf>
- Watts, J. (1995). « A practical approach to dynamic load balancing » [Mémoire de master, California Institute of Technology]. *Caltech THESIS*. https://thesis.caltech.edu/6924/1/Watts_j_1995.pdf
- VMware. (s. d.). « vSphere DRS performance ». *VMware Documentation*.
<https://www.vmware.com/docs/vsphere6-drs-perf>
- Wilcox, T. C. (2008). *Dynamic load balancing of virtual machines hosted on Xen* [Mémoire de master, Brigham Young University]. Brigham Young University - Provo. (pp. 23-30)
- Proxmox VE. (s. d.). « Proxmox VE API ». *Proxmox Documentation*.
https://pve.proxmox.com/wiki/Proxmox_VE_API
- Proxmox VE. (s. d.). « Migration of virtual machines ». *Proxmox Documentation*.
https://pve.proxmox.com/wiki/Migration_of_virtual_machines
- InfluxData. (s. d.). « InfluxDB v2 API documentation ». *InfluxData Documentation*.
<https://docs.influxdata.com/influxdb/v2/>
- InfluxData. (s. d.). « Telegraf Proxmox input plugin ». *Telegraf Documentation*.
<https://github.com/influxdata/telegraf/tree/master/plugins/inputs/proxmox>
- The Go Authors. (s. d.). « The Go Programming Language ». <https://go.dev/doc/>